

Building a Semantic Vector Space in MATLAB

A from-scratch companion to the Node.js embeddings pipeline

LIS 467 — Client/Server & Full-Stack Architectures

October 2026

Building a Semantic Vector Space in MATLAB

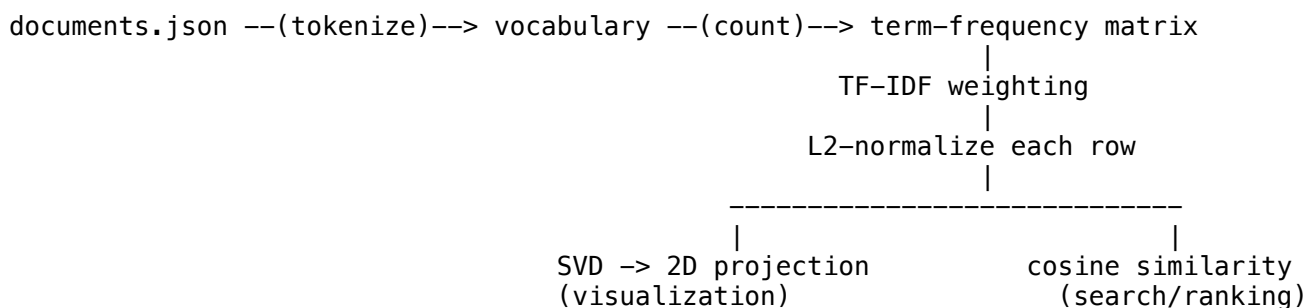
This document walks through `matlab/build_embeddings_demo.m`, a standalone MATLAB script that builds a high-dimensional semantic vector space **from scratch**, using only core MATLAB (no toolboxes). It mirrors every concept in the Node.js demo (`src/embeddings.js`, `src/vectorStore.js`) but trades the neural embedding model for classical TF-IDF vectors, so the linear algebra is fully transparent and inspectable.

1. Why a Separate MATLAB Version?

The Node.js demo uses a pretrained neural network (Xenova/all-MiniLM-L6-v2) to turn text into 384-dimensional vectors. That model is effective, but it's a black box — you can't easily see *why* two pieces of text end up close together in vector space. This MATLAB script rebuilds the same idea with **TF-IDF** (Term Frequency–Inverse Document Frequency), a much older and fully explainable technique, so every number in the resulting vector can be traced back to a specific word and a specific formula. The goal is pedagogical: same architecture, fully visible math.

Reference reading: - MATLAB documentation, `tfidf` / Text Analytics Toolbox overview — <https://www.mathworks.com/help/textanalytics/ref/tfidf.html> - Stanford NLP, *Term weighting and the vector space model* (classic TF-IDF reference) — <https://nlp.stanford.edu/IR-book/html/htmledition/term-frequency-and-weighting-1.html> - MATLAB documentation, `svd` — <https://www.mathworks.com/help/matlab/ref/double.svd.html>

2. Pipeline Overview



Two outputs come from one set of vectors: a **2D scatter plot** (for visualization) and a **ranked search result list** (for a query), exactly paralleling how the Node.js demo's `index.json` vectors feed both the `/api/search` endpoint and could feed a visualization tool.

3. Step 1 — Loading the Documents

```
jsonPath = fullfile(fileparts(mfilename('fullpath')), '..', 'data', 'documents.json');
raw = jsondecode(fileread(jsonPath));
```

MATLAB's built-in `jsondecode` parses the exact same `data/documents.json` file used by the Node.js `scripts/buildIndex.js` — both pipelines share one source of truth. `fileread` plus `jsondecode` is the documented idiom for consuming JSON in MATLAB:

<https://www.mathworks.com/help/matlab/ref/jsondecode.html>.

Title and body text are concatenated (`title + ". " + text`) before tokenizing, for the same reason as in `scripts/buildIndex.js`: a short document's title often carries a large share of its topical meaning.

4. Step 2 — Tokenizing and Building the Vocabulary

```
function tokens = tokenize(txt)
    lowered = lower(char(txt));
    cleaned = regexp(lowered, '^[a-z0-9\s]', ' ');
    parts = strsplit(cleaned);
    tokens = parts(~cellfun('isempty', parts));
end
```

This is a minimal but complete tokenizer: lowercase everything (so “Learning” and “learning” are treated as the same word), strip punctuation with a regular expression, then split on whitespace. `regexp` is MATLAB's regex-replace function — <https://www.mathworks.com/help/matlab/ref/regexp.html>.

The **vocabulary** is built with a `containers.Map`, MATLAB's hash-map/ dictionary type (<https://www.mathworks.com/help/matlab/ref/containers.map.html>), assigning each unique word an integer index the first time it's seen:

```
vocab = containers.Map('KeyType', 'char', 'ValueType', 'double');
...
if ~isKey(vocab, tokens{t})
    vocab(tokens{t}) = vocab.Count + 1;
end
```

This vocabulary **is** the vector space: its size (`vocabSize`) becomes the number of dimensions every document vector will have. Conceptually this is identical to how a neural embedding model fixes its output dimensionality (384 for `all-MiniLM-L6-v2`) — the difference is that here, each dimension has a literal, human-readable meaning: “dimension 57 is the word *neural*,” for example, rather than an opaque learned feature.

5. Step 3 — The Term-Frequency Matrix

```
tf = zeros(numDocs, vocabSize);
for i = 1:numDocs
    tokens = tokensPerDoc{i};
    for t = 1:numel(tokens)
        col = vocab(tokens{t});
        tf(i, col) = tf(i, col) + 1;
    end
end
```

`tf` is a `numDocs × vocabSize` matrix — the classic **bag-of-words** representation. Row `i` is document `i`'s raw vector in the vocabulary's vector space: `tf(i, j)` counts how many times vocabulary word `j` appears in document `i`. At this stage, every document already “lives” in a high-dimensional space (one axis per vocabulary word) — this project's 16-document, ~270-word vocabulary produces 270-dimensional vectors, in the same way the Node.js model produces 384-dimensional vectors, just with a transparent (if much simpler) construction method.

Reference reading: - Wikipedia, *Bag-of-words model* — https://en.wikipedia.org/wiki/Bag-of-words_model

6. Step 4 — TF-IDF Weighting and Normalization

Raw word counts over-weight common words (“the”, “and”) that appear in almost every document and carry little distinguishing meaning. **TF-IDF** corrects this by multiplying each word's count by an *inverse document frequency* factor — words that appear in fewer documents get a higher multiplier:

```
docFreq = sum(tf > 0, 1);
idf = log((numDocs + 1) ./ (docFreq + 1)) + 1;
tfidf = tf .* idf;
```

$$\text{idf}(w) = \ln\left(\frac{N + 1}{\text{df}(w) + 1}\right) + 1$$

where N is the total number of documents and $\text{df}(w)$ is the number of documents containing word w . The $+1$ smoothing terms prevent division-by-zero and avoid a zero IDF for words that appear in every document — this is the same smoothed formula used by scikit-learn's default `TfidfVectorizer` (https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html) and MATLAB's own Text Analytics Toolbox `tfidf` function.

```
docVectors = tfidf ./ vecnorm(tfidf, 2, 2);
```

`vecnorm(tfidf, 2, 2)` computes the L2 (Euclidean) norm of each row — i.e. each document vector's length — and dividing by it rescales every vector to unit length. This is **exactly** the role `normalize: true` plays in `src/embeddings.js`'s call to the `Transformers.js` pipeline. The payoff is identical in both codebases: once vectors are unit length, cosine similarity reduces to a plain dot product (see section 8). `vecnorm` is documented at <https://www.mathworks.com/help/matlab/ref/vecnorm.html>.

7. Step 5 — Dimensionality Reduction with SVD (for Visualization)

A 270-dimensional space can't be plotted directly. **Singular Value Decomposition (SVD)** finds the directions of greatest variance in the data — the same underlying math as **Principal Component Analysis (PCA)** — so the dominant structure of the space can be projected down into 2 or 3 dimensions for plotting, while preserving as much of the original separation between documents as possible.

```
centered = docVectors - mean(docVectors, 1);
[~, S, V] = svd(centered, 'econ');
explainedVar = (diag(S).^2) / sum(diag(S).^2) * 100;
```

```
numComponents = min(3, size(V, 2));
projected = centered * V(:, 1:numComponents);
```

- **Centering** (`docVectors - mean(...)`) shifts the data so it's centered at the origin — a required preprocessing step before SVD/PCA, otherwise the first component would simply capture the overall

mean vector rather than meaningful variance.

- `svd(centered, 'econ')` computes the “economy-size” SVD ($X = USV^T$), returning the singular values (S) and right singular vectors (V) — documented at <https://www.mathworks.com/help/matlab/ref/double.svd.html>. The columns of V are the **principal directions** of the dataset, ordered from most to least variance-explaining.
- `explainedVar` converts squared singular values into a percentage of total variance explained by each component — this is how you answer “how much information did we lose by projecting down to 2D?”
- `projected = centered * V(:, 1:numComponents)` performs the actual projection: multiplying the centered data by the top components maps every 270-dimensional document vector down to a 2D (or 3D) point.

Reference reading: - MATLAB documentation, *Principal Component Analysis (PCA)* —

<https://www.mathworks.com/help/stats/pca.html> (this script implements the same idea manually via `svd` rather than calling `pca` directly, to keep the Statistics Toolbox out of the dependency list) - 3Blue1Brown, *Essence of linear algebra: SVD intuition* (visual explanation) — <https://www.3blue1brown.com/topics/linear-algebra>

The resulting plot

```
figure('Name', 'Semantic Vector Space (reduced to 2D)', 'Color', 'w');
...
scatter(projected(mask, 1), projected(mask, 2), 80, colors(c, :), 'filled', ...);
```

Each document becomes a point; color encodes its category (Computer Science, History, Cooking, Gardening, Library Science). When run, this plot visually confirms the central claim of the whole project: **documents about similar topics cluster together in vector space**, even though the only information fed into the model was raw word counts — no human ever told the algorithm “these are both about computers.” The two Computer Science documents in the sample dataset land near each other and distinctly apart from the Cooking and Gardening documents purely as an emergent property of the TF-IDF + SVD math.

8. Step 6 — Embedding a Query and Ranking by Cosine Similarity

```
queryVec = queryVec .* idf;
norm_q = norm(queryVec);
if norm_q > 0
    queryVec = queryVec / norm_q;
end

scores = docVectors * queryVec';
[sortedScores, order] = sort(scores, 'descend');
```

The query string goes through the **identical** pipeline as every document: tokenize → count against the existing vocabulary → multiply by the same `idf` weights computed from the document collection → L2-normalize. This consistency is critical — a query must be embedded with the *same* vocabulary and weighting scheme as the documents, or the resulting vector would not be comparable to them at all. This mirrors `src/server.js`, which calls the exact same `embedText()` function for both indexing and query time.

Because both `docVectors` rows and `queryVec` are unit-length, the matrix multiplication `docVectors * queryVec'` computes the dot product between the query and *every* document vector simultaneously — a vectorized, one-line equivalent of the `cosineSimilarity()` loop in `src/vectorStore.js`. This is the same mathematical shortcut explained in `DEEP_DIVE.md` section 4.2, and `sort(scores, 'descend')` then ranks documents exactly like `semanticSearch()`'s `.sort((a, b) => b.score - a.score)` in the Node.js code.

Reference reading: - Wikipedia, *Cosine similarity* — https://en.wikipedia.org/wiki/Cosine_similarity

9. TF-IDF vs. Neural Embeddings: What's the Same, What's Different

Aspect	This MATLAB script (TF-IDF)	Node.js demo (all-MiniLM-L6-v2)
Vector dimensionality	Size of vocabulary (~270 here, grows with corpus)	Fixed at 384, set by the model architecture
What each dimension means	A specific word	An opaque learned feature (not human-readable)
Captures synonyms?	No — “car” and “automobile” are unrelated dimensions	Yes — trained on massive text corpora to place synonyms nearby
Captures word order / context?	No (bag-of-words)	Yes (transformer attention mechanism)
Computational cost	Trivial — counting and matrix algebra	Higher — neural network forward pass
Transparency	Fully inspectable — every number traces to a word	Black box — can't easily explain <i>why</i> two vectors are close

TF-IDF is a reasonable *first step* into “vectors that represent meaning,” and it genuinely does capture topical similarity (as the clustering plot shows). Neural embeddings go further by also capturing synonymy, word order, and deeper semantic relationships — which is why the “zero shared words” demo queries in the main project (DEMO_SCRIPT.md) work so well with the neural model but would be harder for pure TF-IDF to match (TF-IDF relies on at least some literal word overlap, since every dimension is a specific word).

10. Summary

This script rebuilds, with nothing but core MATLAB matrix operations, the same three ideas that underlie the production Node.js demo:

1. **Text** → **high-dimensional vector** (tokenize + count + TF-IDF, vs. a neural network forward pass)
2. **Normalize vectors so cosine similarity becomes a dot product** (vecnorm division, vs. `normalize: true` in Transformers.js)
3. **Rank by similarity** (matrix multiply + sort, vs. the `cosineSimilarity()` loop in `vectorStore.js`)

The SVD-based 2D visualization is the one piece with no direct counterpart in the Node.js demo — it exists purely to make the abstract idea of a “vector space” visually concrete for a classroom audience.